IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA
(Deemed to be University)

YENEPOYA (DEEMED TO BE UNIVERSITY)

Project - Low Level Design

on

Predictive Modeling for Retail Success

Submitted by

Muhammed Shahabaz T M

23BCDSBA67

Industry Mentor:

Mr.Ujjwal Kumar

Guided by:

Ms.Raksha Adyanthaya

TABLE OF CONTENTS

S.No	Content	Page No
1	Introduction	3
1.1	Scope of the Document	4
1.2	Intended Audience	4
1.3	System Overview	4
2	System Design	5
2.1	Application Design	6
2.2	Process Flow	6
2.3	Information Flow	7
2.4	Components Design	7
2.5	Key Design Considerations	8
2.6	API Catalogue	8
3	Data Design	9
3.1	Data Model	9
3.2	Data Access Mechanism	10
3.3	Data Retention Policies	10
3.4	Data Migration	10
4	Interfaces	11
5	State and Session Management	13
6	Caching	15
7	Non-Functional Requirements	17
7.1	Security	17
7.2	Performance	18
8	References	19

1 INTRODUCTION

This document provides a detailed design of the predictive modeling system developed for retail analysis. It explains the internal working of the system, including data processing, model building, and prediction mechanisms.

The purpose of this document is to describe how the system is implemented using machine learning techniques. It gives a clear understanding of the steps involved in transforming raw retail data into meaningful insights.

The system uses Python and various libraries to process data, train models, and generate predictions. This detailed design helps in understanding the functionality and performance of the system.

1.1 Scope of the Document

This document covers the detailed implementation of the predictive modeling system developed for retail sales prediction. It explains the internal working of the system, including data processing steps such as data loading, cleaning, transformation, and feature selection. The document also describes how the machine learning model is executed, trained, and used to generate predictions. Overall, it provides a clear understanding of how the system is built and how it functions at a detailed level.

1.2 Intended Audience

This document is intended for developers, students, and evaluators who are interested in understanding the internal design and implementation of the system. Developers can use this document as a reference for building or modifying the system. Students can understand how the project is implemented step by step, and evaluators can review the technical details and working of the system.

1.3 System Overview

The system is designed to predict retail sales using a machine learning approach. It uses the Linear Regression algorithm to analyze historical data and generate predictions. The system takes input features such as age, quantity, and price, and processes them to estimate the total sales amount. The overall goal of the system is to provide accurate sales predictions, helping in better decision-making and analysis in a retail environment.

2 SYSTEM DESIGN

The system design describes the detailed working process of the predictive modeling system used for retail analysis. It explains how the data is processed and how the model is built step by step.

Initially, the data is loaded using the Python pandas library. The dataset is read from a CSV file using the read_csv() function. After loading the data, preprocessing is performed to prepare the dataset for analysis. This includes removing missing values and converting data into appropriate formats.

In the next stage, important features such as Age, Quantity, and Price are selected for building the predictive model. The dataset is then divided into training and testing sets, where 80% of the data is used for training and 20% is used for testing.

The system uses the Linear Regression algorithm for prediction. The model is trained using the training dataset with the help of the function model.fit(X_train, Y_train). Once the model is trained, predictions are generated using model.predict(X_test).

This structured design ensures that the system processes data efficiently and produces accurate predictions for retail sales analysis.

2.1 Application Design

The application is designed to process retail data and generate sales predictions using a machine learning model. The input to the system is a dataset in CSV format containing various attributes related to retail transactions. The processing stage includes multiple steps such as data cleaning, feature selection, and model training. During data cleaning, missing or inconsistent values are handled to ensure data quality. Feature selection is performed to choose relevant input variables such as age, quantity, and price. These selected features are then used to train the machine learning model. The output of the system is the predicted sales values, which are generated after the model processes the input data.

2.2 Process Flow

The system follows a structured step-by-step process to generate predictions. Initially, the dataset is loaded using the pandas library, and the CSV file is read using the read_csv() function. Once the data is loaded, missing values are identified and handled appropriately to maintain data quality. The system then converts data types where necessary to ensure consistency in processing. Relevant features such as age, quantity, and price are selected for model training. The dataset is then split into training and testing sets, typically in an 80% to 20% ratio. The Linear Regression model is trained using the training data. After training, the model is tested using the testing dataset to evaluate its performance. Finally, the system generates predictions based on the trained model and displays the results.

2.3 Information Flow

The information flow in the system follows a linear sequence. The dataset is first provided as input and then passed through preprocessing steps where it is cleaned and prepared. The processed data is then fed into the machine learning model. The model analyzes the data and generates predictions, which are finally displayed as output. This flow ensures a clear and organized movement of data from input to final result.

2.4 Components Design

The system is divided into several functional components to ensure modularity and clarity. The Data Input Module is responsible for loading the dataset into the system. The Preprocessing Module handles tasks such as data cleaning, transformation, and feature selection. The Model Training Module is used to train the Linear Regression model using the prepared data. The Prediction Module generates the final output by applying the trained model to the test data. Each component works together to ensure smooth execution of the system.

2.5 Key Design Considerations

Several important factors are considered while designing the system. Accuracy of the model is a key consideration, as the system aims to generate reliable sales predictions. Simplicity of implementation is also important to ensure that the system is easy to understand and maintain. Efficiency is considered to ensure that the system processes data quickly and produces results without significant delay.

2.6 API Catalogue

The system uses various libraries to perform different tasks. The Pandas library is used for data handling, including loading, cleaning, and manipulating the dataset. The Scikit-learn library is used to implement the Linear Regression model and perform machine learning tasks such as training and prediction. Matplotlib can be optionally used for data visualization, although it is not mandatory for the system.

3 DATA DESIGN

The data design describes the structure of the dataset used in the predictive modeling system. The dataset contains various attributes related to retail transactions, which are used for analysis and prediction.

The dataset includes the following fields:

- Transaction ID: A unique identifier for each transaction.
- Customer ID: A unique identifier for each customer.
- Age: Represents the age of the customer.
- Product Category: Specifies the category of the product purchased.
- Quantity: Indicates the number of items purchased in a transaction.
- Price: Represents the price of a single unit of the product.
- Total Amount: The total value of the transaction, calculated based on quantity and price.

These data attributes are used as input for the predictive model to analyze patterns and generate accurate predictions. Proper data design ensures efficient data handling and improves the performance of the system.

3.1 Data Model

The data model used in this system consists of several key attributes that are essential for predicting retail sales. Each record in the dataset represents a transaction and includes fields such as Transaction ID and Customer ID for identification purposes. Additional attributes such as age, quantity, and price are used as input features for the prediction model. The total amount represents the target variable that the system aims to predict. This structured data model helps in organizing the dataset and ensures that relevant information is available for processing and analysis.

3.2 Data Access Mechanism

The system uses a CSV file as the primary data source. The dataset is loaded into the system using the pandas library, which provides efficient methods for reading and handling structured data. The read_csv() function is used to import the dataset into the program, allowing easy access and manipulation of the data during processing.

3.3 Data Retention Policies

The system follows a simple data retention approach where the data is used temporarily during execution. The dataset is loaded into memory, processed, and used for model training and prediction. Once the execution is complete, the data is not stored permanently within the system. This ensures simplicity and avoids unnecessary data storage.

3.4 Data Migration

Data migration is not required in this system, as the dataset is directly loaded and used within the application. There is no need to transfer data between different systems or databases. The system operates on a single dataset, making the process straightforward and efficient.

4 INTERFACES

The interface of the predictive modeling system plays an important role in defining how users interact with the system and how the results are presented. In this project, the interface is designed to be simple, clear, and easy to use. The system is implemented using Jupyter Notebook, which provides an interactive environment for executing code, analyzing data, and displaying outputs in an organized manner.

Users interact with the system through the Jupyter Notebook interface. The notebook consists of multiple code cells, each performing a specific task in the predictive modeling process. The user executes these cells step by step to perform operations such as loading the dataset, cleaning the data, selecting relevant features, training the model, and generating predictions. This step-by-step execution helps users clearly understand how the system works internally and allows them to track the output at each stage.

The input to the system is provided in the form of a CSV file containing retail transaction data. This dataset includes attributes such as transaction ID, customer ID, age, quantity, price, and total amount. The user loads this dataset into the system using the pandas library. Once the data is loaded, it becomes available for processing and analysis within the notebook environment.

The system processes the input data and produces output in the form of tables. These tables are displayed directly in the Jupyter Notebook after executing the corresponding code cells. The output tables include dataset previews, cleaned data, selected features, and the predicted values generated by the machine learning model. The tabular format ensures that users can easily read and interpret the results without confusion.

The interface is designed to maintain clarity and organization. Each step of the process is separated into different sections, and the results are displayed immediately after execution. This improves readability and helps users identify any issues or errors during execution. The integration of code and output in a single interface makes the system more transparent and easy to understand.

Another advantage of using Jupyter Notebook is its flexibility. Users can modify the code, experiment with different inputs, and rerun the model to observe changes in the output. This feature is particularly useful for testing and improving the predictive model. It also allows users to learn and explore different aspects of data analysis and machine learning.

The system does not require a complex graphical user interface, as all interactions are handled within the notebook. This keeps the system lightweight and easy to use. Since the project focuses on predictive modeling for retail success, the interface is designed to support efficient data analysis rather than visual complexity.

In conclusion, the interface of the system is simple, interactive, and effective. By using Jupyter Notebook and presenting outputs in table format, the system ensures ease of use and clear communication of results. This design supports users in understanding the predictive modeling process and helps in analyzing retail sales data efficiently.

5 STATE AND SESSION MANAGEMENT

State and session management refers to how a system maintains information during its execution and across multiple interactions. In many applications, session management is used to track user activities, maintain login sessions, and store temporary data between different operations. However, in the predictive modeling system for retail success, state and session management is not required due to the simple and standalone nature of the system.

The system is implemented using Jupyter Notebook, where execution is performed step by step through individual code cells. Each time the notebook is run, the system starts from the beginning and processes the data independently. There is no need to store or maintain information between different executions, as each run is treated as a fresh instance.

The input dataset is loaded into the system during execution using the pandas library. Once loaded, the data is processed through various steps such as cleaning, feature selection, model training, and prediction. All these operations take place within a single execution cycle. After the execution is completed, the results are displayed, and no information is retained for future runs unless explicitly saved by the user.

Since the system does not involve multiple users or continuous interaction, there is no requirement for session tracking. There are no login mechanisms, user authentication processes, or user-specific data storage. This eliminates the need for maintaining session states and simplifies the overall system design.
The stateless nature of the system ensures that each execution is independent and does not depend on previous results. This improves reliability, as the output is always generated from the current input data and logic. It also makes debugging easier, as errors can be traced within a single execution without considering previous states.

Another advantage of not using session management is efficient memory usage. Since no session data is stored, the system only uses memory during execution. Once the notebook run is completed or the kernel is restarted, all temporary data is cleared automatically. This helps in maintaining system performance and prevents unnecessary memory consumption.

In the context of predictive modeling for retail success, this design is appropriate because the system primarily focuses on analyzing data and generating predictions in a single run. The user can rerun the notebook whenever needed to obtain updated results. This approach keeps the system simple, efficient, and easy to understand.

In conclusion, the predictive modeling system does not require state or session management. It operates as a stateless system where each execution is independent.

6 CACHING

Caching is a technique used in many systems to store frequently accessed data or previously computed results in temporary storage, so that future requests can be processed faster. It helps improve performance and reduces processing time by avoiding repeated computations. However, in the predictive modeling system for retail success, caching is not implemented due to the simple structure and nature of the application.

The system is developed using Jupyter Notebook, where execution is carried out step by step. Each time the notebook is run, the dataset is loaded, processed, and analyzed from the beginning. Since the dataset used in this project is relatively small and the processing time is minimal, there is no significant benefit in storing intermediate results in cache memory. Therefore, the system directly processes the data during each execution.

In this project, the retail dataset is stored in a CSV file and loaded using the pandas library. All data processing steps such as cleaning, transformation, feature selection, and model training are performed dynamically during runtime. The machine learning model is trained and tested within the same execution, and predictions are generated immediately. Since these steps are executed only once per run, caching is not necessary.

Another important reason for not using caching is to ensure that the results are always accurate and up to date. By processing the data from scratch each time, the system avoids the risk of using outdated or incorrect cached data. This approach guarantees that the predictions are based on the latest input data and processing logic.
The absence of caching also simplifies the system design. There is no need to manage additional storage for cached data or implement mechanisms to update and invalidate cache entries. This makes the system easier to develop, maintain, and understand, especially for academic purposes.
From a performance perspective, the system performs efficiently even without caching. The use of optimized libraries such as Pandas and Scikit-learn ensures that data

processing and model execution are completed quickly. The system runs smoothly on standard hardware without requiring advanced optimization techniques.

In the context of predictive modeling for retail success, the focus is on understanding the workflow and generating accurate predictions rather than optimizing performance through caching. Therefore, the decision to exclude caching aligns with the goals of the project.

In conclusion, caching is not used in this system, and all data is processed directly during each execution. This approach ensures simplicity, accuracy, and efficient resource usage while maintaining the overall effectiveness of the predictive modeling system.

7 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes of the system such as security, performance, reliability, and efficiency. These requirements ensure that the predictive modeling system for retail success operates smoothly and provides a good user experience. Unlike functional requirements, which describe what the system does, non-functional requirements focus on how well the system performs its tasks.

In this project, the system is designed to be simple, efficient, and reliable while processing retail data and generating accurate predictions. Since the application is implemented using Jupyter Notebook and operates on a local dataset, the design focuses on maintaining data safety and ensuring fast execution without unnecessary complexity.

7.1 Security

Security is an important aspect of any system, even for a simple academic project. In this predictive modeling system, data security is maintained by using local data storage. The dataset is stored in a CSV file on the user's system and is not transmitted over any network. This reduces the risk of unauthorized access or data breaches.

Basic data handling practices are followed to ensure data integrity. The system does not involve user authentication, login mechanisms, or external database connections, which minimizes security risks. Since the data is processed locally within the Jupyter Notebook environment, it remains under the control of the user.
In addition, no sensitive personal information is exposed or shared. The system is designed only for analysis and prediction purposes, ensuring that the data is used responsibly and securely. This approach keeps the system simple while maintaining an adequate level of security for the project.

7.2 Performance

Performance refers to how efficiently the system processes data and generates results. The predictive modeling system is designed to provide fast execution and quick response times. Since the dataset is relatively small and the operations are not computationally intensive, the system performs efficiently even on standard hardware.

The use of lightweight libraries such as Pandas and Scikit-learn ensures that data processing and model execution are carried out quickly. The Linear Regression algorithm used in the system is simple and efficient, which contributes to faster training and prediction.

The system processes data directly during execution without using caching or complex optimization techniques. This approach ensures consistent performance and avoids unnecessary overhead. Users can run the system multiple times without experiencing delays, making it suitable for testing and demonstration purposes.

Overall, the system provides reliable and efficient performance, ensuring that predictions are generated quickly and accurately. This makes it effective for analyzing retail data and supporting decision-making.

8 REFERENCES

The development of this predictive modeling system for retail success is supported by various tools and libraries. Python is used as the primary programming language due to its simplicity and powerful support for data analysis and machine learning. The Pandas library is used for data handling, including loading, cleaning, and manipulating the dataset. The Scikit-learn library is used for implementing the Linear Regression algorithm and performing machine learning tasks such as training and prediction. These tools provide an efficient and reliable foundation for building the system.